FRANKENSTEIN · PITCH-PREP PACKET · AI ENGINEER CODE SUMMIT · SAT NOV 7 2026 · JUDGING AFTER 17:00

Lost games, rebuilt and proven.

THE ONE LINE JUDGES SHOULD REPEAT

> **“The agents rebuilt Doom’s code from the cartridge, and the compiler proved every piece.”**

THE PROBLEM

87% of classic games released in the US are critically endangered (Video Game History Foundation and Software Preservation Network, 2023). For most of them only the compiled cartridge survives. It runs, but nobody can fix it, change it or move it, because the source code is gone.

WHAT WE BUILT TODAY

A crew of AI agents that reads a game’s machine code and writes the C source back, one function at a time. A referee compiles every piece with the original compiler and accepts it only if the bytes are identical. Then we splice the agents’ code back into the game and the whole cartridge checksums identical. First patient: GBADoom.

235 / 568
game functions rebuilt (41.4% of the functions)

8.39%
of code bytes: 11,900 of 141,774. The small ones came first.

IDENTICAL
rebuild: 235 agent-written + 333 original functions

0.1–0.2 s
per check by the referee

39
lies caught (agents claimed a match that wasn’t one)

32 / 0
fakes rejected / fakes accepted

0 of 20
compiled when Muse worked from memory alone: it isn’t recall

200%
health from god mode in the changed build (`god_200`)

Rebuild SHA-1 `d936259bcbe48b5ce5d8be4d5ffe935c1eece8b2` · 36 finished runs · numbers frozen at 16:12, commit `b805d42` (DECISIONS.md 16:11 and 16:12).

SPONSORS ARE ORGANS, NOT LOGOS

SPONSOR	ROLE	WHAT IT DOES ON SCREEN
Meta · Muse Spark, Muse Wonder	The hands	Writes every line of recovered C.
Subquadratic · SubQ Snap	The judgment	Triage, escalate or abandon, fakematch screening, the Stage director, the “eyes” that confirm a change on screen.
Fetch.ai · ASI:One, Agentverse	The front door	Anyone asks for proof in chat. The Excavator agent runs a job, then asks the Referee agent agent-to-agent before it reports success.
MongoDB Atlas	The memory	Every habit the crew learns and every event lives in Atlas.
Google Gemini	Second opinion	The second lab that screens every match.

The 3-minute judging-room script

02 · based on docs/DEMO_SCRIPT_V2.md

KARTHIK speaks the whole pitch AIDENN drives the keys and the Stage ENZO phones, controller, the game

EDDY referee and technical Q&A

Open in advance: the Stage Split tab `http://127.0.0.1:5173/?legacy=1&scene=split` and the labelled REPLAY tab `http://127.0.0.1:5173/?legacy=1&scene=split&replay=doom_r01&speed=4`. Start the live redo with `make demo-live ARGS="--redo 2"`. Stop at 3:00, even mid-sentence.

TIME	ON SCREEN	WHO ACTS	EXACT WORDS (KARTHIK)	FALLBACK
0:00 **-0:12**	Doom running full screen.	KARTHIK ENZO game up	"For most old games, only the cartridge survives. The recipe, the source code, is gone. It's like being handed a cake and asked for the exact recipe. Our agents write the recipe back from the cartridge, and a compiler bakes it to check it's the same cake, byte for byte. This is Doom."	No playable ROM: start on the Stage Split scene and say the same words.
0:12 **-0:20**	Terminal: `Judge: press ENTER to hire the agents.` running `make demo-live ARGS="--redo 2"`.	AIDENN sets it up · JUDGE presses Enter	"Press Enter. You just hired two agents. With 235 already back, watch it redo one from scratch: library off, no head start."	Referee down: "The referee is down, so nothing can count; here's the last real run." Aidenn switches to the REPLAY tab (REPLAY · 4x badge stays visible).
0:20 **-0:55**	Stage Split: machine code left, Muse writing C right, the MATCH stamp, then the judgment rail.	AIDENN Stage · KARTHIK	"Left: what's on the cartridge. Right: C that Meta's Muse Spark is writing right now. One rule: it only counts if it compiles to the exact same bytes." At MATCH: "There. Byte for byte, checked in a fifth of a second. From memory alone, Muse compiled 0 of 20, so this isn't recall. Then SubQ Snap screens it for tricks."	**Nothing lands by 0:50:** Aidenn switches to the REPLAY tab. Karthik: "Agents are slow; the referee isn't. Here's a real run from this morning." First MATCH in under 2 s.
0:55 **-1:15**	Versus or Road scene: a claimed match, the referee's re-run, REJECTED with the first wrong instruction.	AIDENN scene · KARTHIK	"Agents lie. Today they told us "it matches" 39 times when it didn't. The referee re-runs every claim, and one instruction gives each of them away. Agents lie. Compilers don't."	No live lie in Versus: show the FALSE CLAIM card on `/judges` (a real event from `doom_r12`).
1:15 **-1:35**	Ending: ORIGINAL and REBUILT checksums, IDENTICAL. Then the meter.	AIDENN · KARTHIK	"We put the agents' code back into Doom: 235 functions. Same checksum as the original. Not similar: identical. That's 235 of Doom's 568 functions so far, the small ones first. And every run starts from the last one's best try; that's how a 90% near-miss became a match."	Ending says NOT REBUILT YET: show the `/judges` rebuild card.
1:35 **-2:00**	Doom (original build), controller in the judge's hands.	ENZO hands controller · JUDGE presses	"Doom's god-mode cheat on the Game Boy is a button sequence. Press up, up, down, down, left, left, right, right. That just ran the god-mode function our agents rebuilt from machine code; the code that reads the buttons is still the original."	Cheat doesn't trigger: "The god-mode function is one we rebuilt." Move on. (Tested 15:07 in the browser player: HEALTH 100%.)

TIME	ON SCREEN	WHO ACTS	EXACT WORDS (KARTHIK)	FALLBACK
2:00 –2:30	Same cheat in the changed build: status bar reads **200%**. Optional: Snap's eyes line "CHANGE CONFIRMED".	**ENZO** swaps build · **JUDGE** presses	"And this is why source matters. An emulator can only play the game. We changed one line in the agents' god-mode C, rebuilt, and now god mode gives you 200% health. Everything else is byte for byte the same."	Use a fresh session (saved god-mode state skews it). `god_200` won't boot: cut the beat, say "Recovered code can be changed and rebuilt; that's next." (`instant_raise` is untested; don't use it.)
2:30 –2:45	**IF READY** Game picker on the laptop, Doom preselected → QR → phone loads → Game Boy controls take shape → playable Doom.	**ENZO** phones · **JUDGE** scans	"Pick a game. Scan it. That's our rebuilt game on your phone, running in a browser emulator, with controls made for your hands."	Not shown working in Lane D's files (see note below). No QR: Enzo hands over Phone A. No phone works: skip straight to the close.
2:45 –3:00	Credits card: Built today / Not ours.	**KARTHIK**	"Ask for any function in ASI:One and the Fetch.ai agents bring back the proof. Muse writes it, Snap judges it, the compiler proves it. On the left, everything we built today; on the right, everyone else's work. Lost games, rebuilt and proven."	None. Stop at 3:00.

Why the phone beat is marked IF READY. `creature/RESULTS_DEMO.md` does not exist. `creature/EXPERIENCE_IDEA.md` describes the picker, QR and adaptive controls as "product requirements and research directions, not completed features". `creature/DEMO_READINESS.md` and `creature/STATUS.md` say the phone bundle is built but not deployed, and physical phones, the public judging URL and the QR are still pending. Run it only if Enzo has shown it working on a real phone before 17:00.

The film closing

03 · last lines of the 1-minute submission film

VOICE: KARTHIK · OVER THE GOD-MODE CHEAT, HEALTH 100% BESIDE 200%

“Now that we have the code, we can change it.

We changed one line of the AI’s code.

Same cheat, double the health.

Not emulated. Not imitated. Rebuilt, byte for byte.”

Frankenstein.

IF A JUDGE HAS SEEN THE FILM

- Lead with the live redo, not the film’s shots: the judge pressing Enter is what scores Technical Demo.
- Repeat the same anchor line; don’t introduce a new one.
- The REPLAY badge stays on screen whenever we show recorded runs.

CREDITS WE MUST SHOW

Built today: the agents, the referee, the Stage, the patches, the phone page and the site.
Not ours: Doom, GBADoom, the shareware WAD, mGBA, devkitARM and the partner platforms.

ONE NUMBER PER SCENE

SCENE	THE ONE NUMBER	SOURCE
The dig	**0.2 s**	A check takes 0.1 to 0.2 s (DECISIONS.md 11:24)
The lie	**39**	Lies caught, `tools/counts.py` (DECISIONS.md 16:12)
The rebuild	**235 of 568**	Union rebuild, identical, commit b805d42
The change	**200%**	`god_200`, tested 15:07, `lab/demo/shots/god_200_after.png`

Q&A prep

04 · the biggest section · one breath each

Answer first
Then one piece of evidence. Then stop.

One number
Only numbers from the repo. Say "of the functions".

Admit limits early
Say what isn't done before they ask. It scores as confidence.

Tie back
End on preservation: a game you can fix, change and move.

Default owner: EDDY for referee and technical questions, KARTHIK for everything else, ENZO for the phone and game, AIDENN for the Stage.

TECHNICAL

QUESTION	ANSWER	SOURCE
How does matching work? EDDY	An agent sees one function's machine code under an anonymous id and writes C. The referee compiles it with the game's own compiler and flags and compares the bytes. Identical, it's a MATCH; one byte off, it isn't. 0.1 to 0.2 s per check.	DECISIONS 11:24
Why byte-identical instead of "behaves the same"?	Because it's a proof, not an opinion. Tests can miss a branch; identical bytes can't. And identical bytes mean the whole cartridge checksums identical when we put the code back.	README
How do you know it isn't memorized? GBADoom is public.	We scrambled our build with small edits and hid the source. From memory alone, Muse compiled 0 of 20. Even with the real headers and compiler errors, 3 of 20 matched, none of them functions we'd changed.	DECISIONS 13:28 · lab/out/memory_probe.json
How is the full rebuild proven? EDDY	Object splice: the referee re-checks every recovered file (each must MATCH), writes the agents' own compiled bytes into the ROM, and keeps linked addresses only where it proved them identical. 235 agent-written plus 333 original functions: SHA-1 identical, d936259b…, verified on two machines.	DECISIONS 11:49, 16:11, 16:12
Couldn't an agent cheat the compiler, like inline assembly?	That's a fakematch, and we screen for it: the Checker's rules plus SubQ Snap. 32 fakes rejected, 0 accepted. The final union was re-checked against the fakematch rules with 0 failures.	DECISIONS 16:12
Why only 8.39% of the code if you have 41% of the functions?	Small functions came first. What's left is big: the renderer and the status-bar face. That's 235 of 568 functions but only 11,900 of 141,774 code bytes. The next lever is bigger functions, not more runs.	DECISIONS 16:12

QUESTION	ANSWER	SOURCE
What's hard about the remaining 59%?	Size and compiler habits. On big functions one register choice moves every byte after it, so "almost right" is still wrong. Our permuter tried exact rewrites on 17 near-misses at 90% or better and closed none.	DECISIONS 15:31 · crew/EXPERIMENTS.md
Which compiler and flags?	devkitARM r68, GCC 16.1.0 in Docker, GBADoom's own Makefile flags minus link-time optimisation, at -O2. Pinned by digest.	DECISIONS 11:20 · lab/TOOLCHAIN.md
How would you do another game?	Point the same referee at a new target and compiler. We tried Attack on Voxelburg: 4 of 20 on a first run. We don't claim Doom's library made it faster: one run each, no measured transfer.	DECISIONS 13:43 · lab/TRANSFER.md
Does the recovered code run anywhere else?	Six recovered functions compiled for this Mac and for WebAssembly gave the same outputs as the original machine code: 20,000 of 20,000 identical in a browser page. It says nothing about the other 562 functions, and it isn't a port.	lab/demo/equivalence/EQUIVALENCE.md

TRUST BUT VERIFY

QUESTION	ANSWER	SOURCE
What does “lie” mean?	An agent said “it matches” and it didn’t. 39 times today. The referee re-ran each claim and showed the first wrong instruction.	`tools/counts.py` · DECISIONS 16:12
And fakes?	Code that passes the byte check by a trick instead of real C. 32 rejected, 0 accepted, across 36 finished runs.	same
How do I re-verify a claim myself? EDDY	Ask in ASI:One. The Excavator agent asks the Referee agent agent-to-agent and only answers with its verdict. Or run `python3 tools/counts.py`: the tallies come only from files in the repo.	DECISIONS 12:46, 15:06
Why trust the referee?	It doesn’t think, it compiles. Same input, same bytes, every time, and anyone can re-run it from the repo. Agents lie. Compilers don’t.	`contracts/`

THE SYSTEM THAT GETS BETTER

QUESTION	ANSWER	SOURCE
Does it get better?	Run over run, yes: warm starts and salvage turned near-misses into matches; one 90% near-miss became a MATCH. On held-out code, no measured gain yet: library on versus off, 29 vs 29 of 40 easy, 6 vs 6 of 20 hard. So we don’t claim one.	README · `crew/EXPERIMENTS.md`
Then why build the library?	Because the gain we can see is on the hard near-misses, and that’s where the remaining code is. Measuring it honestly on held-out code is the next experiment, not a slide.	`crew/EXPERIMENTS.md`

BUSINESS AND IMPACT

QUESTION	ANSWER	SOURCE
Who pays for this?	Rights holders re-releasing old games, studios that lost their own source, and the archives and preservation groups that keep games alive. Source is what lets a game be fixed, ported and re-released.	`site/judges.html`
How big is the problem?	87% of classic games released in the US are critically endangered (Video Game History Foundation and Software Preservation Network, 2023). For most, only the cartridge survives.	README
Why not just emulate?	An emulator can only play a game. With source you can fix it, change it and move it. You just saw it: one line changed, 200% health.	`god_200`
What’s next?	The big functions, then a second real game. Today the phone runs our rebuilt ROM in an emulator; a WebAssembly or native build from recovered C is the step after.	`DEMO_SCRIPT_V2`

LEGAL AND ETHICS

QUESTION	ANSWER	SOURCE
Is this piracy?	No commercial ROMs, ever. GBADoom is GPL and we use the shareware WAD. The point is the opposite of piracy: rights holders are the customers.	DECISIONS (target)
Did the agents see the source?	Never. They see anonymous ids and machine code. GBADoom's public source is our answer key, which is why we picked it.	DEMO_SCRIPT_V2

SPONSORS

QUESTION	ANSWER	SOURCE
Where is Meta?	Muse writes every line of recovered C. Muse Spark digs; when Snap escalates a stubborn function, Muse Wonder takes it.	README · DECISIONS
Where is Subquadratic?	SubQ Snap is the judgment: triage, when to escalate or abandon, fakematch screening, the Stage director and the eyes that confirm the change on screen.	CLAUDE.md · README
Where is Fetch.ai?	The front door. Anyone can ask for proof in ASI:One. The Excavator runs a job and asks the Referee agent before it reports, agent to agent. Team `aie-ff9bcbc6`, 4 of 4 joined. Shared chat: `asi1.ai/shared-chat/5dfc14f7-815e-42e7-a0be-96db539cf5ab`	DECISIONS 15:52, 16:04
Where is MongoDB?	Every habit and every event lives in Atlas.	DEMO_SCRIPT_V2
Where is Gemini?	The second lab that screens every match.	DEMO_SCRIPT_V2

HACKATHON GENERIC

QUESTION	ANSWER	SOURCE
What did you build today versus before?	Everything runnable was written after 10:45 today. Only the planning docs in `docs/` and the hackathon clock were prior work.	CLAUDE.md
What was hardest?	Making “byte-identical” real: pinning the exact compiler, and catching agents that say they matched when they didn’t. 39 times.	DECISIONS
Biggest failure?	Our permuter: exact rewrites on 17 near-misses at 90% or better closed none. And held-out code shows no measured gain yet.	DECISIONS 15:31
What surprised you?	How often agents lie about success, and how cheaply a compiler catches it: one instruction gives each one away.	tools/counts.py
How would you scale it?	The referee is a 0.2 s check, so the cost is model time, not proof. More agents in parallel on more functions; every match is independent and re-checkable.	DECISIONS 11:24
Why this team?	Four lanes, one repo: Eddy built the referee and lab, Karthik the crew, Aidenn the Stage, Enzo the game and phones. Each of us can answer for our own part.	CLAUDE.md
“It’s just Doom.”	The game looks the same on purpose: that’s the proof. What we built is the system that wrote 235 of its functions back from machine code and the referee that refuses anything that isn’t byte-identical. You just ran one.	DEMO_SCRIPT_V2

Never say

- ~~"The whole game recovered"~~ or ~~"Doom recovered"~~ → "235 of Doom's 568 functions"
- ~~Any percentage without "of the functions"~~ → "41.4% of the functions, 8.39% of the code bytes"
- ~~"Port"~~ for the phone → "our rebuilt ROM, running in a browser emulator"
- ~~"Dashboard"~~ → "the Stage"
- Any number not in this packet or the repo, any held-out gain, any transfer to the second game
- ~~"The face is back"~~ or anything about the renderer being recovered

Q&A rules drawn from: Hackathon survival guide: questions every team should be ready to answer (dev.to) · What judges actually score (dev.to) · Deliver a winning pitch at hackathons (Salesforce). They agree on concise answers, stating limits before a judge raises them, and being able to explain your own core work. Numbers: DECISIONS.md 16:11 and 16:12 (commit b805d42), README.md, crew/EXPERIMENTS.md, lab/demo/equivalence/EQUIVALENCE.md.